

정규식을 활용한 입력값 필터링

01 입력값 필터 만들기



학습 목표

- ✅ 정규식을 활용하여 입력값을 필터링 하는 필터를 구현한다
- ✅ Spring Boot 프레임워크에 필터를 적용한다.
- ✅ XSS 대응을 위한 필터를 적용하여 취약점을 제거한다.



학습 내용

1. 정규식으로 입력값을 필터링 할 수 있는 필터 제작
2. SpringBoot 프레임워크에서 필터를 적용하여 입력값 필터링
3. XSS 취약점의 기본적인 이해 및 필터링 적용



정규식을 활용한 입력값 필터링

01 입력값 필터 만들기

필터(Filter)란?

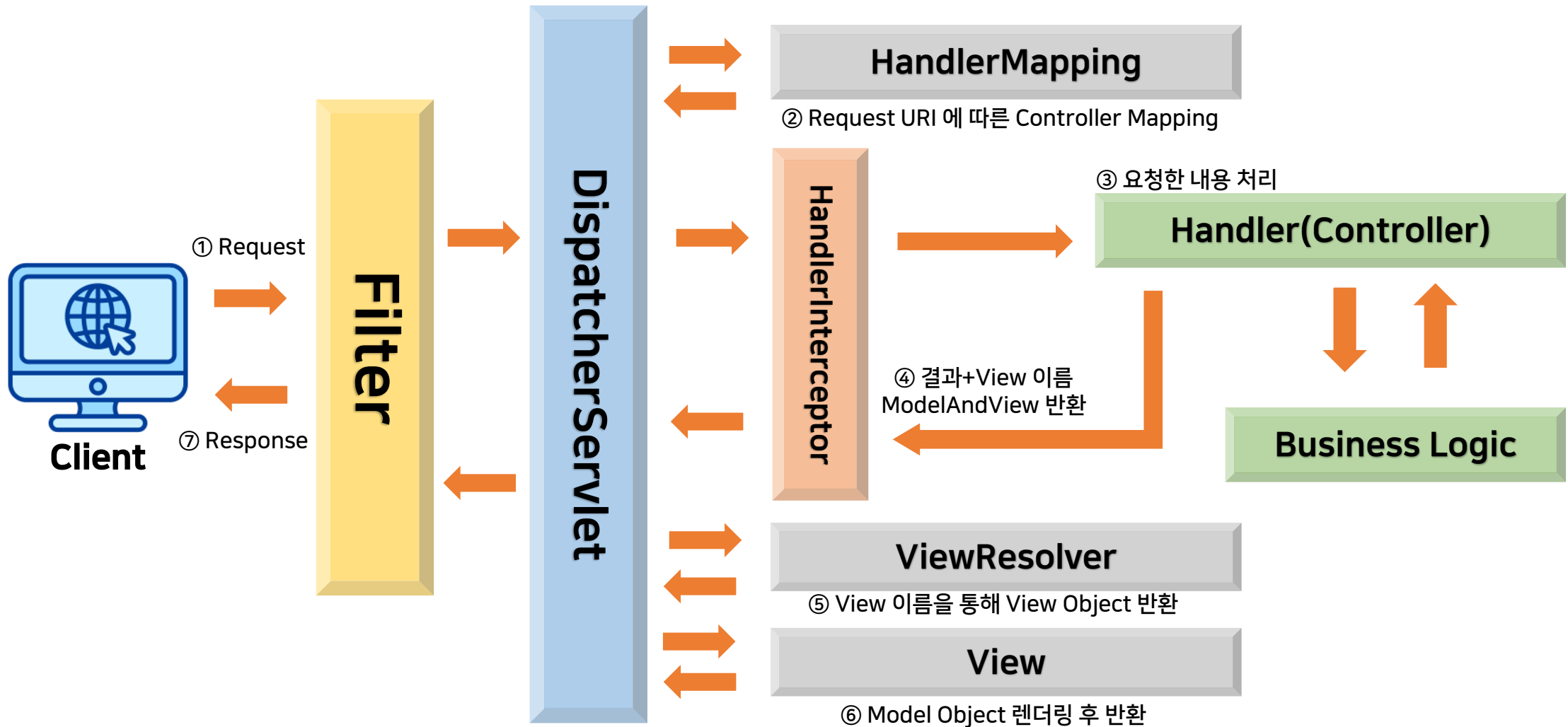
필터(Filter)란?

클라이언트의 HTTP 요청과 응답을 가로채어 특정 작업을 수행하는 서블릿 기반 컴포넌트. 웹 애플리케이션에서 보안, 로깅, 데이터 변환등의 작업을 수행할 때 사용

필터의 주요 역할

- 요청/응답 로깅
- 인증(Authentication) & 권한 검사(Authorization)
- CORS 처리(다른 도메인의 요청 허용 또는 차단)
- 데이터 검증 & 변환(요청 데이터 가공 및 유효성 검사)
- 응답변환(JSON,XML 등 특정형식으로 변경)
- 보안강화(SQL Injection, XSS 등의 공격 방어)

Spring Boot Filter 구조



Spring Boot 에서 필터 구현 방법

STEP1. 기본 필터 구현

```
public class CustomFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        System.out.println("Request URI: " + httpRequest.getRequestURI());

        // 다음 필터 또는 컨트롤러로 요청 전달
        chain.doFilter(request, response);
    }

    @Override
    public void destroy() {
    }
}
```

Spring Boot 에서 필터 구현 방법

STEP2. 필터를 Spring Boot 에 등록

(방법1) @Component를 사용하여 자동 등록

```
@Component
public class CustomFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        System.out.println("Request Method: " + httpRequest.getMethod());

        chain.doFilter(request, response);
    }
}
```

Spring Boot 에서 필터 구현 방법

STEP2. 필터를 Spring Boot 에 등록

(방법2) FilterRegistrationBean을 사용하여 수동 등록(우선순위 지정이 필요한 경우)

FilterConfig

```
@Configuration
public class FilterConfig {

    @Bean
    public FilterRegistrationBean<CustomFilter> loggingFilter() {
        FilterRegistrationBean<CustomFilter> registrationBean = new FilterRegistrationBean<>();
        registrationBean.setFilter(new CustomFilter());
        registrationBean.addUrlPatterns("/api/*"); // 특정 URL 패턴에만 적용
        registrationBean.setOrder(1); // 필터 실행 순서 지정 (낮을수록 먼저 실행)
        return registrationBean;
    }
}
```

Spring Security 와 기본 필터

Spring Security 사용 시 자동으로 보안 필터 적용

Spring Security에는 로그인, 인증, 권한체크 등을 처리하는 필터가 포함되어 있음.

DefaultSecurityConfig

```
@Configuration
public class DefaultSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated() // 모든 요청 인증 필요
            )
            .formLogin() // 기본 로그인 폼 활성화
            .httpBasic(); // HTTP Basic 인증 활성화

        return http.build();
    }
}
```

입력값 필터 만들기

XSS (Cross-Site Scripting) 공격이란

게시판처럼 입력 내용 그대로 표출되는 곳에 악성 스크립트를 넣어, 서비스 이용자의 브라우저에서 실행되도록 하는 공격

XSS 공격 방어 전략

- 사용자 입력값에서 위험한 패턴의 문자열을 필터링
- SpringBoot의 `ServletFilter`와 `HttpServletRequestWrapper`를 이용하여 모든 요청의 입력값을 가로챈
- 입력값의 내용을 정규식으로 패턴을 확인하여 필터링 처리



입력값 필터 만들기

XSS 필터 구현

모든 HTTP Request를 XSSRequestWrapper로 감싸서 모든 요청 내용의 입력 값 필터링하도록 함

```
1 package kr.co.openeg.lab.filter.xss;
2
3 import javax.servlet.*;
4
5
6
7
8 @Component
9 public class p043_XSSFilter implements Filter {
10
11     @Override
12     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
13         throws IOException, ServletException {
14         HttpServletRequest httpRequest = (HttpServletRequest) request;
15         chain.doFilter(new p043_XSSRequestWrapper(httpRequest), response);
16     }
17 }
18
```

p043_XSSFilter.java

입력값 필터 만들기

XSSRequestWrapper 구현

```
1 package kr.co.openeg.lab.filter.xss;
2
3 import javax.servlet.http.HttpServletRequest;
4
5 public class p043_XSSRequestWrapper extends HttpServletRequestWrapper {
6
7     public p043_XSSRequestWrapper(HttpServletRequest request) {
8         super(request);
9     }
10
11     @Override
12     public String getParameter(String name) {
13         String value = super.getParameter(name);
14         return (value != null) ? sanitize(value) : null;
15     }
16
17     @Override
18     public String[] getParameterValues(String name) {
19         String[] values = super.getParameterValues(name);
20         if (values != null) {
21             for (int i = 0; i < values.length; i++) {
22                 values[i] = sanitize(values[i]);
23             }
24         }
25         return values;
26     }
27 }
28
```

```
29 // XSS 공격 가능성이 있는 코드 제거
30 private String sanitize(String input) {
31     if (input == null) return null;
32     String clean = input;
33     // <script> 태그 제거
34     clean = clean.replaceAll("(?i)<script.*?>.*?</script>", "");
35     // onclick, onerror 등의 이벤트 핸들러 제거
36     clean = clean.replaceAll("(?i)on\\w+\\s*=\\s*['\"](?:[^\"]|\"'\")*['\"]", "");
37     // javascript: 문자열 제거
38     clean = clean.replaceAll("(?i)javascript:", "");
39     return clean;
40 }
41 }
42
```

p043_XSSRequestWrapper.java

- 1) <script> 태그와 그 사이의 내용을 제거
- 2) onclick, onerror 등의 이벤트 핸들러 제거
- 3) javascript: 문자열 제거

입력값 필터 만들기

정규식을 이용한 입력값 필터링 정책

- (1) `<script.*>.*</script>`
- (2) `<script.*?>.*?</script>`
- (3) `(?i)<script.*?>.*?</script>`
- (4) `(?si)<script[^>]*>.*?</script>`

- (5) `(?i)on\w+\s*=\s*['\"][^'\"]*['\"]`

- (6) `("(?i)javascript:`

LAB1 - 소스코드의 XSS 취약점 확인

1. 환경 구축한 LAB을 실행시켜 파이어폭스로 로그인

접속 URL : localhost:8000

로그인 계정 : test / qq111

2. '게시판' 메뉴 진입

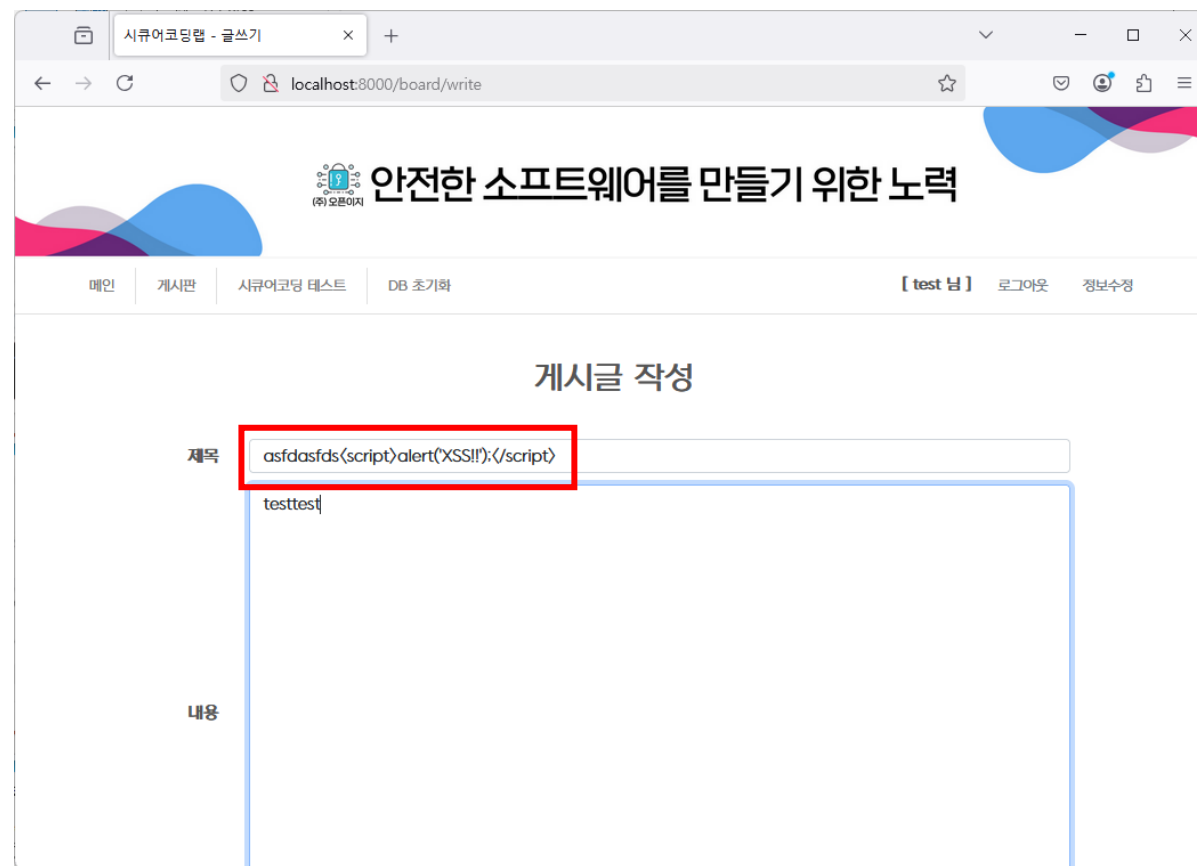
3. '쓰기' 버튼을 눌러 게시글 작성

4. 제목 부분에 아래와 같이 입력

스크립트 구문 외에는 아무 문자열이나 입력해도 됨

안녕하세요<script>alert('XSS!!');</script>

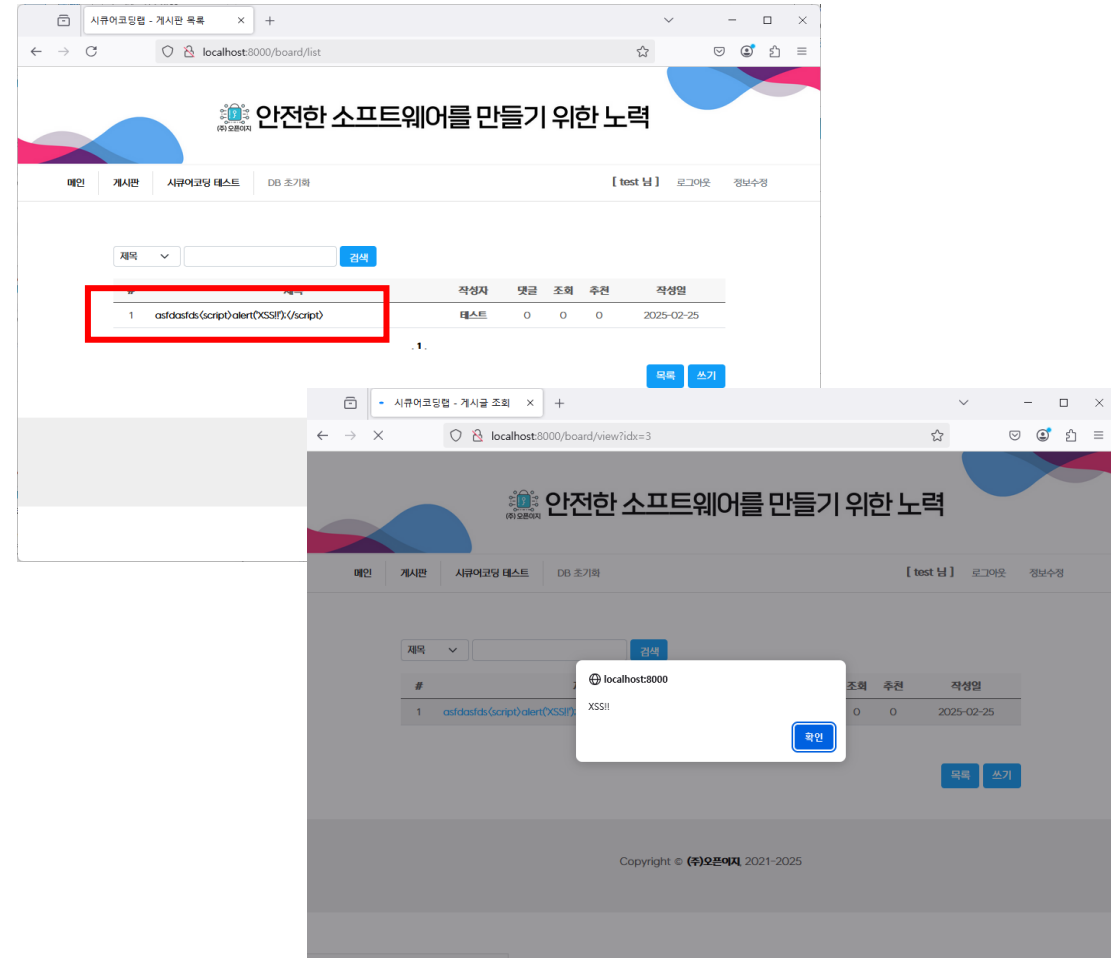
5. 화면 아래로 내려가 '확인' 버튼



LAB1 - 소스코드의 XSS 취약점 확인

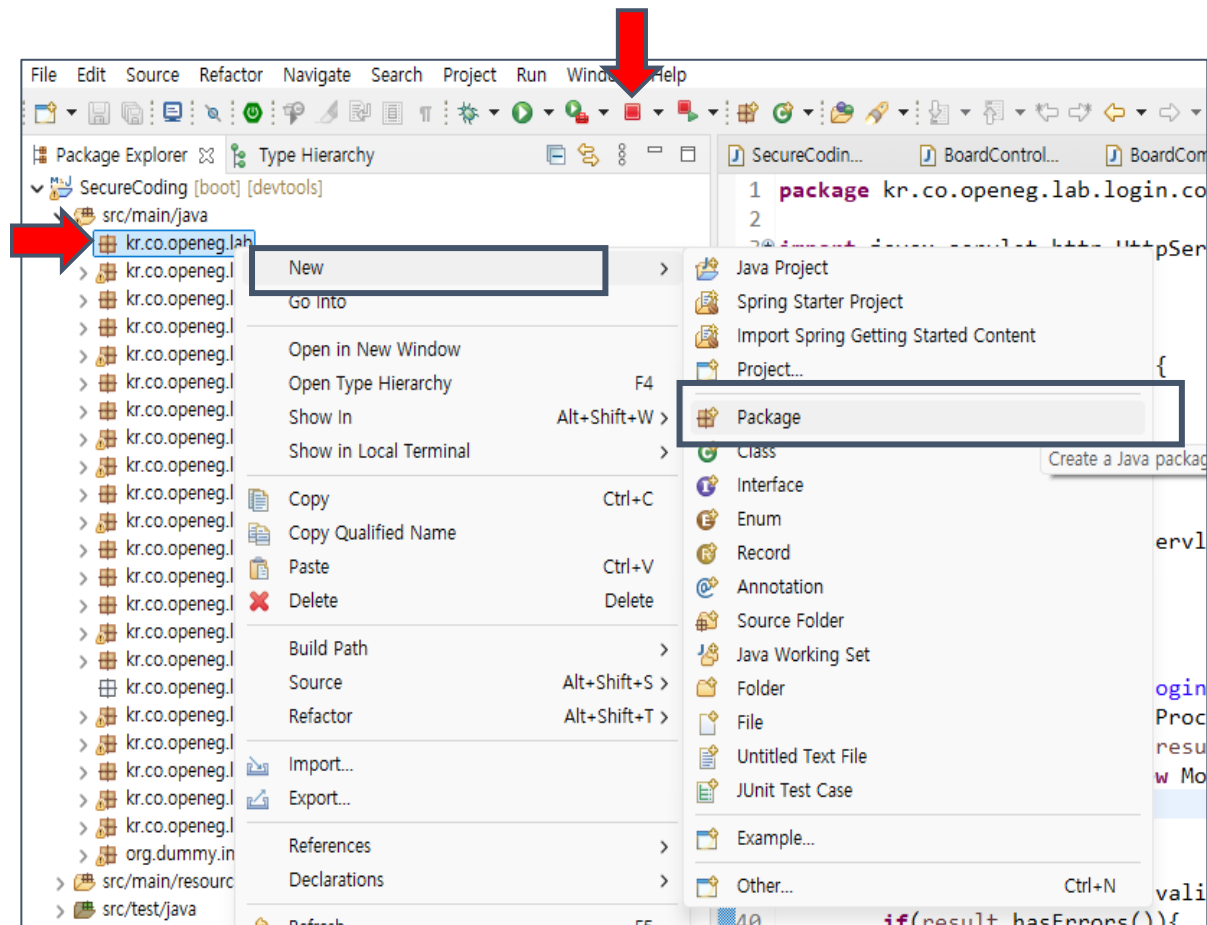
6. 게시판 목록 화면에서 해당 게시글 제목에 스크립트 내용이 보여짐
7. 해당 게시글을 클릭하면 스크립트가 동작하여 화면에 경고창이 나타남

XSS 취약점 있는 것이 확인됨



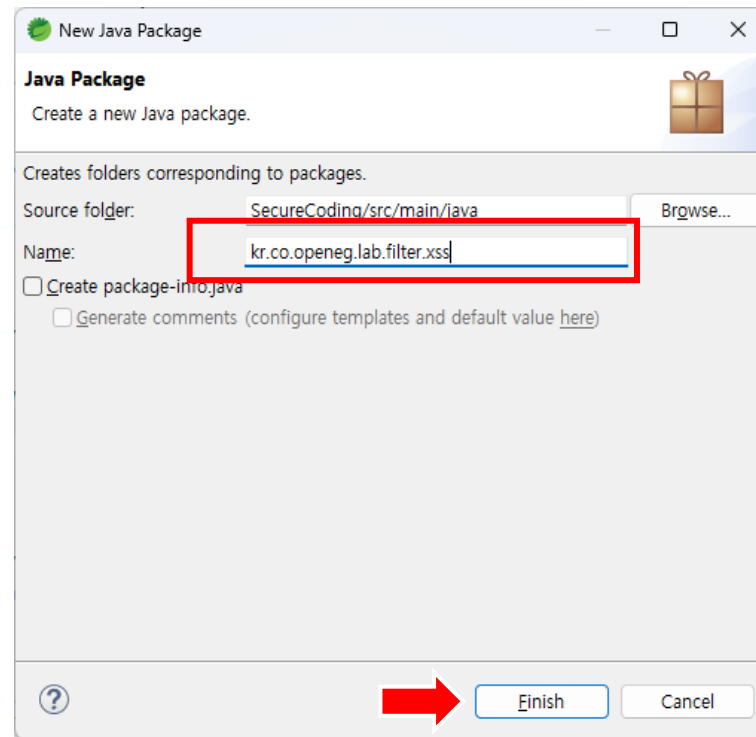
LAB2 - 소스코드에 XSS 필터 반영

1. 서비스 실행 중이라면, 상단 ■ 버튼을 눌러 중지
2. STS의 좌측 Package Explorer에서
kr.co.openeg.lab을 찾아 마우스 오른쪽 버튼 클릭
3. New > Package 선택



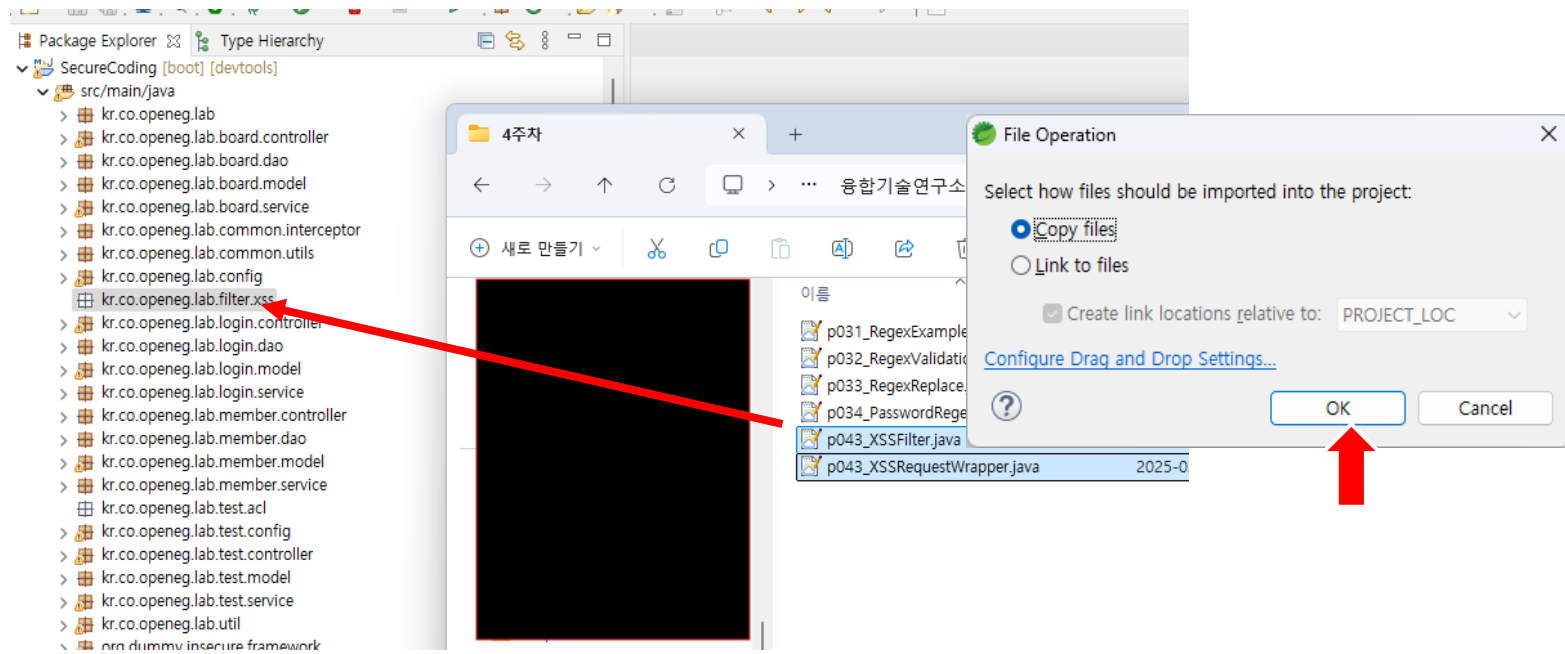
LAB2 - 소스코드에 XSS 필터 반영

3. Java Package 창에서 Name 항목에
kr.co.openeg.lab.filter.xss 입력
4. Finish 버튼



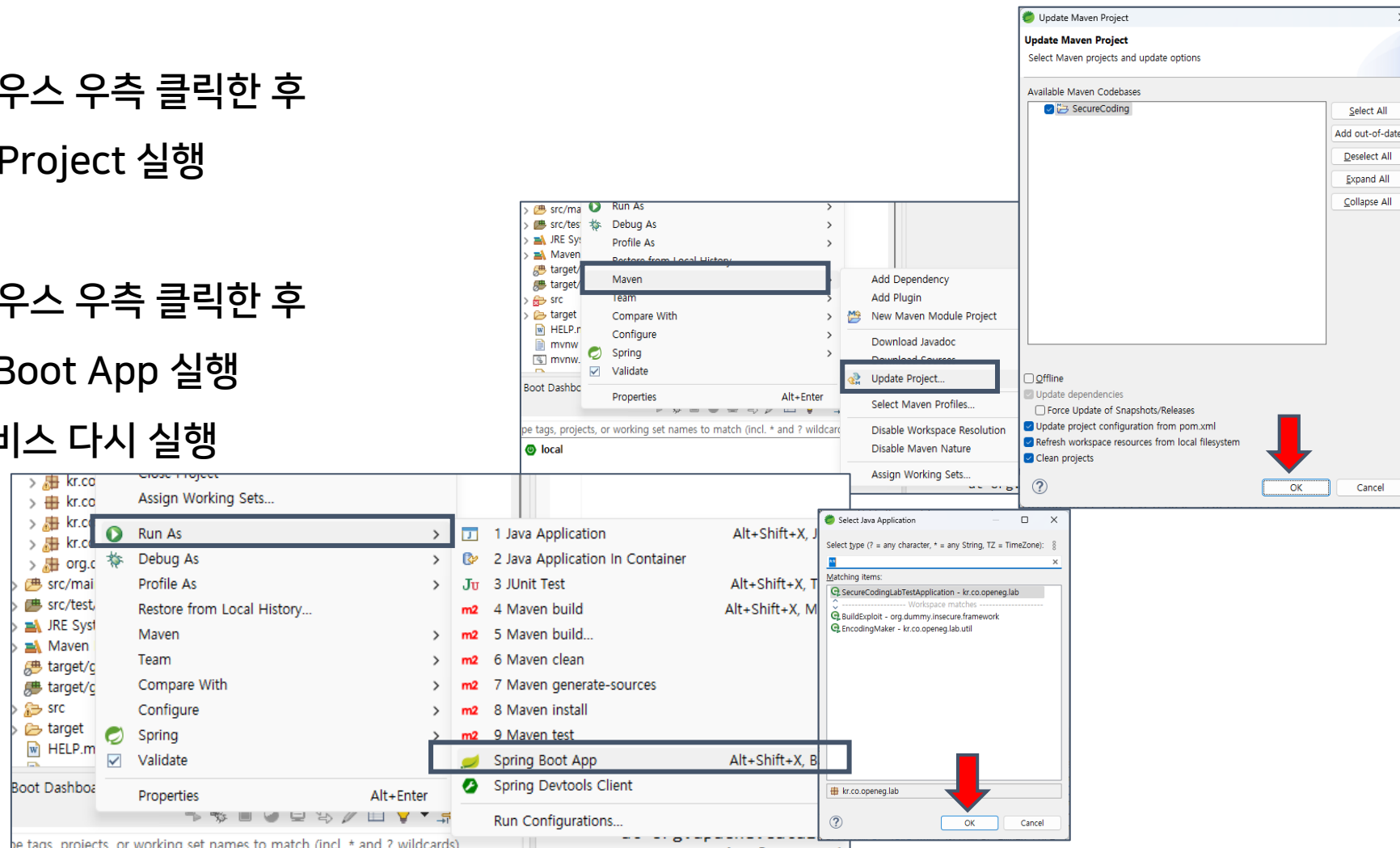
LAB2 - 소스코드에 XSS 필터 반영

5. 폴더에서 XSS 필터 소스코드 2개를 선택한 후, 새롭게 생성하여 비어있는 패키지로 Drag & Drop
6. 파일 Import 방법으로 'Copy files' 선택한 후 OK버튼



LAB2 - 소스코드에 XSS 필터 반영

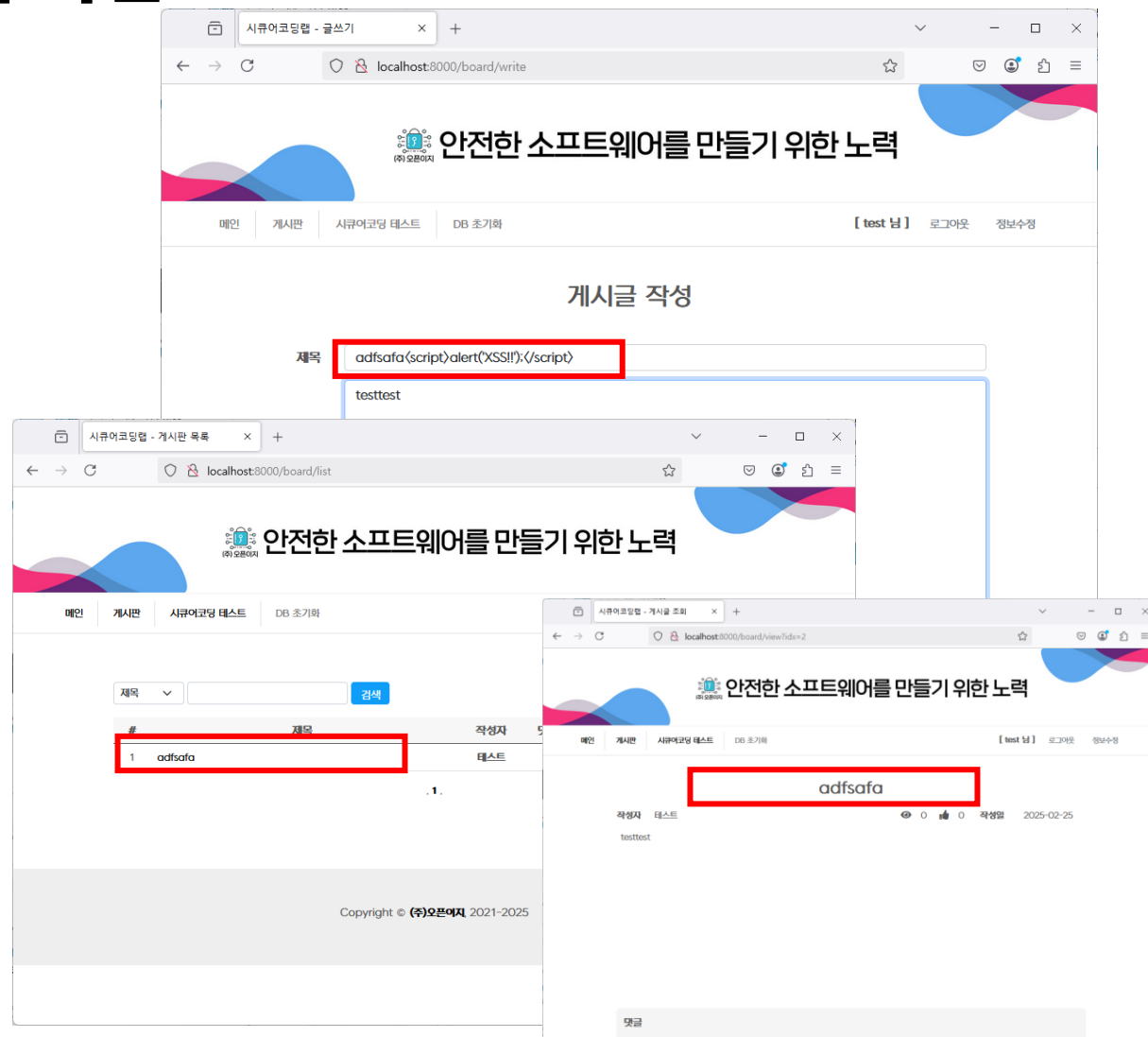
7. 프로젝트 명에서 마우스 우측 클릭한 후
Maven > Update Project 실행
8. 빌드 완료 확인
9. 프로젝트 명에서 마우스 우측 클릭한 후
Run As > Spring Boot App 실행
10. OK 버튼을 눌러 서비스 다시 실행



LAB3 - 소스코드에서 XSS 필터 동작 확인

1. 파이어폭스에서 서비스 재 접속 및 로그인
2. 게시판에 들어가서 마찬가지로 새 글 작성 및 제목에 스크립트 구문 입력
3. 확인 버튼 눌러 글 등록 후 목록에서 제목 확인
제목에 스크립트 부분이 제외된 것이 확인됨
해당 글을 눌러 들어가도 경고창이 나타나지 않고 글 내용이 나타남.

XSS 필터가 글 등록 직전 정상적으로 동작하여 스크립트 구문을 삭제한 것이 확인됨



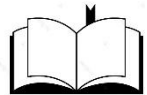
시연 영상

- XSS 필터 만들기



요약 정리

- 필터는 클라이언트의 HTTP 요청과 응답을 가로채어 특정 작업을 수행하는 서블릿 기반 컴포넌트로 웹 애플리케이션에서 보안, 로깅, 데이터변환등의 작업을 수행할 때 사용
- Spring Security 를 설치하면 로그인, 인증, 권한체크 등을 처리하는 필터가 기본적으로 적용
- 웹애플리케이션의 XSS 취약점을 방어하기 위해 XSS Filter를 생성해서 프레임워크에 적용



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the top left, a smaller striped sphere in the top right, and another striped sphere in the bottom left. Several network-like structures, consisting of white dots connected by thin white lines, are scattered across the image. There are also solid blue rectangular bars of different sizes and orientations. The overall aesthetic is modern and technological.

감사합니다.



퀴즈

- 클라이언트의 HTTP 요청과 응답을 가로채어 특정 작업을 수행하는 서블릿 기반 컴포넌트는 무엇인가?
 - Dispatcher
 - Handler
 - Filter
 - ViewResolver
- String 클래스에서 정규식으로 지정한 문자열을 다른 문자열로 바꾸기 위해 사용하는 메서드 이름은?
(영문 10글자, . 제외)
()
- 다음 중 XSS 입력값 필터에서 처리해야 하는 내용이 아닌 것은?
 - <script> 태그와 그 사이 내용을 제거
 - javascript: 문자열 제거
 - onclick, onerror 등의 이벤트 핸들러 제거
 - 압축파일 안에 들어있는 스크립트 파일 제거



퀴즈 정답

1. 클라이언트의 HTTP 요청과 응답을 가로채어 특정 작업을 수행하는 서블릿 기반 컴포넌트는 무엇인가?

2) Filter

2. String 클래스에서 정규식으로 지정한 문자열을 다른 문자열로 바꾸기 위해 사용하는 메서드 이름은?
(영문 10글자, . 제외)

(`replaceAll`)

3. 다음 중 XSS 입력값 필터에서 처리해야 하는 내용이 아닌 것은?

4) 압축파일 안에 들어있는 스크립트 파일 제거

압축파일을 다루는 작업은 필터에서 처리하기에 매우 많은 리소스를 사용하며, XSS 공격 방어와도 관련이 없습니다.